Introduction to Deep Learning & Neural Networks with Keras by IBM

What are the applications of Deep Neural Networks?

- **Colour Restorations**: Automatic colourisation and colour restoration in black and white images. (Made by Japanese)

- **Speech Reenactment**: Synching lip movement in a video with an audio clip. (Made in University of Washington)

- **Automatic Handwriting Generation**: We can rewrite any content in other handwriting formats. (Made in University of Toronto)

- **Automatic Machine Translation**: CNNs are used to translate text in an image on the fly.

- **Automatically adding sounds to Silent Movies**: Using a database of pre-recorded sounds to select a sound to play that best matches the scene.

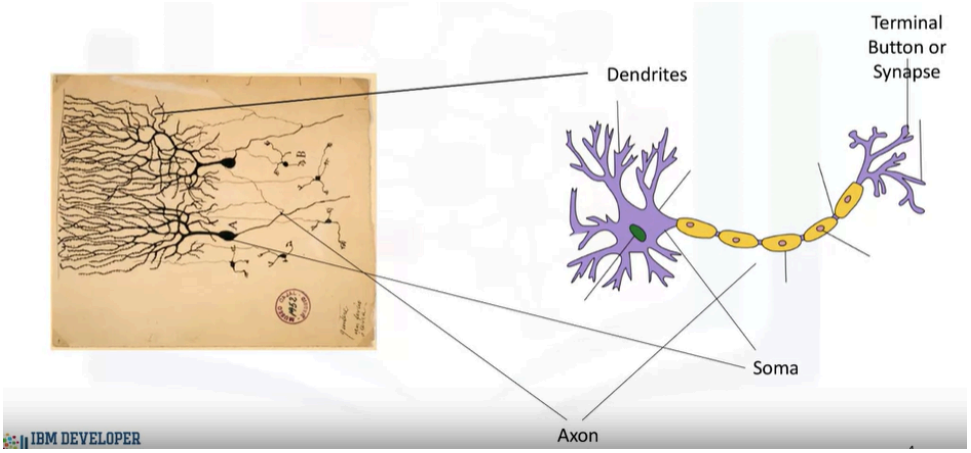
- **Object Classification and Detection in Images**

- **Self Driving Cars**

Neural Network

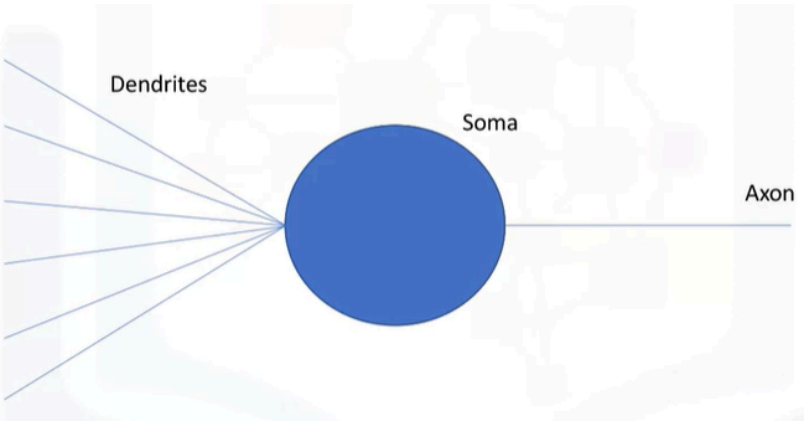
All the algorithms that are used in **Deep Learning** are largely inspired by the way Neurons and Neural Networks function and process data in the Brain.

Remember the structure of the Neurons from Biology.

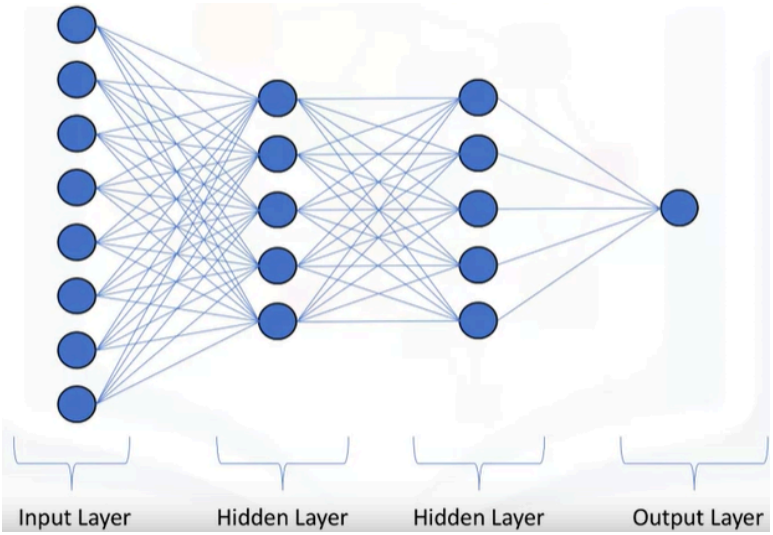


Artificial Neuron

An **Artificial Neuron** behaves in the same way as a **Biological Neuron**.



Artificial Neural Network

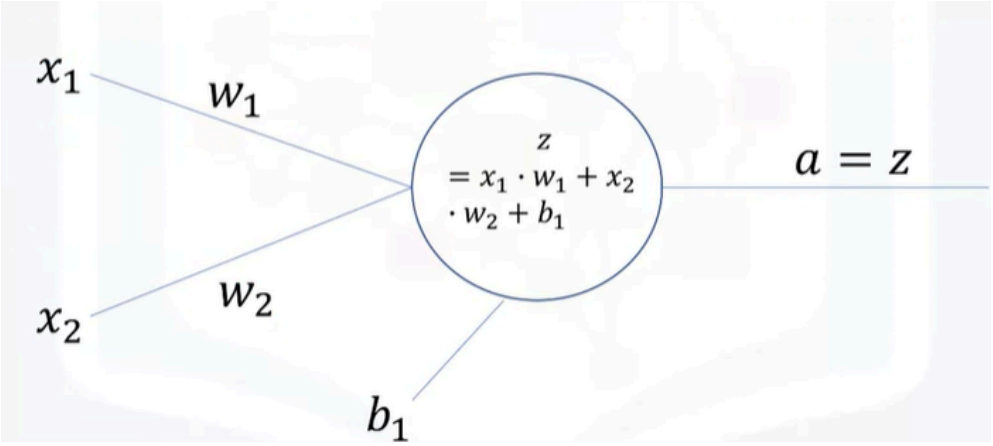


The **first layer** that feeds the input into the network is called the **Input Layer**. The set of nodes that provide the network output is called **Output Layer**. Any set of nodes in between the **Input** and the **Output Layers** are called the **Hidden Layers**.

Three main topics associated with Artificial Neural Networks:

1. **Forward Propagation**
2. **Back Propagation**
3. **Activation Functions**

Forward Propagation



It is the process through which data passes through layers of neurons in a **Neural Network** from the **Input Layer** to the **Output Layer**. The data flows through each neuron by connections or the dendrites. Every connection has a specific weight by which the flow of data is regulated.

Here, $\mathbf{x_1}$ and $\mathbf{x_2}$ are two inputs, they could be an integer or float. When these inputs pass through the connections, they're adjusted depending on the connection weights, $\mathbf{w_1}$ and $\mathbf{w_2}$. The neuron then processes this information by outputting a weighted sum of these inputs. It also adds a constant to the sum which is referred to as the bias. So, **z** is the linear combination of the inputs and weights along with the bias, and **a** is the output of the network.

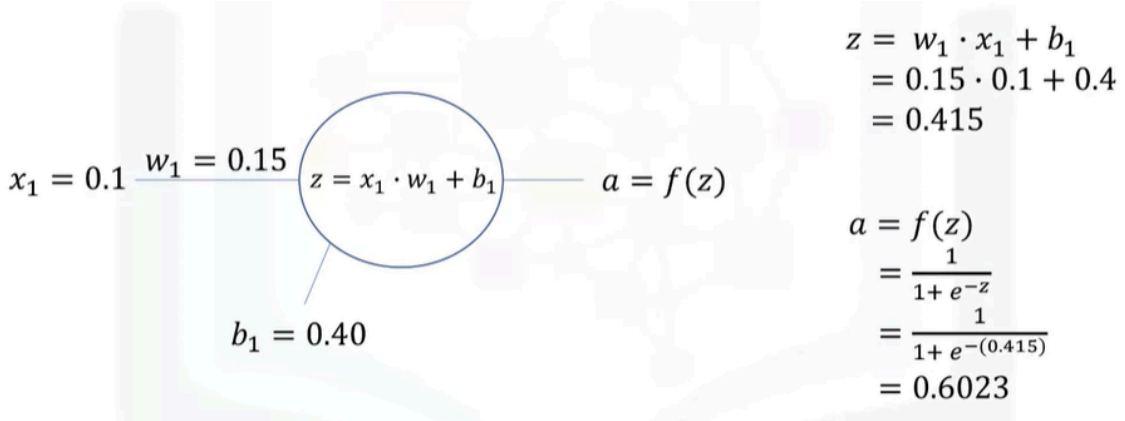
$$a = f(z) = \frac{1}{1 + e^{-z}}$$

However, simply outputting a weighted sum of the input limits the task that the **Neural Network can perform**. Therefore, a better way to process the data would be to map the weighted sum to a non-linear space. A popular function is the **Sigmoid Function**, where if the weighted sum is a very large positive number, then the output of the neuron is close to 1, and if the weighted sum is a very large negative number, then the output of the neuron is close to zero. They are also called **Activation Functions**, they decide

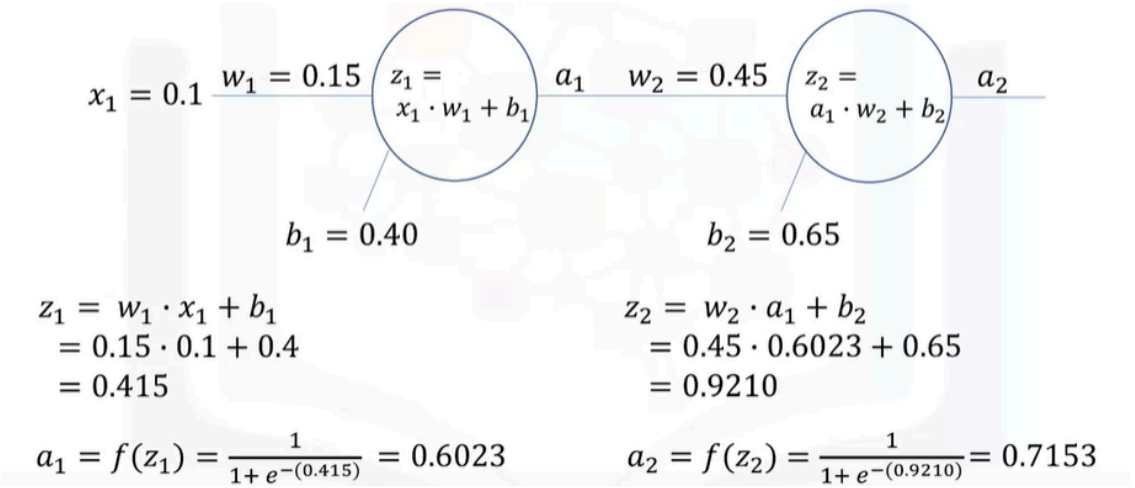
whether a neuron should be activated or not.

A **Neural Network** without an **Activation Function** is nothing but a **Linear Regression Model**. The **Activation Function** performs a non-linear transformation to the input enabling the **Neural Network** to learn and perform more complex tasks, image classification and language translation.

Example of computation in a **Single Neuron**.



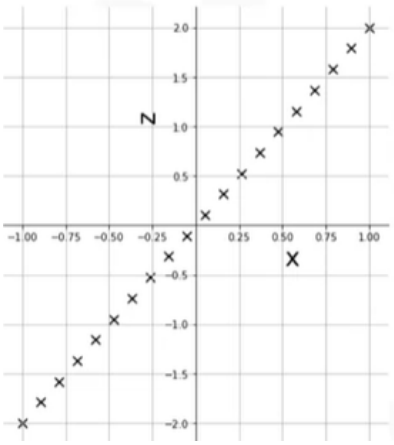
Example of computation in **Two Neurons**.



<u>Gradient Descent</u>

Cost Function

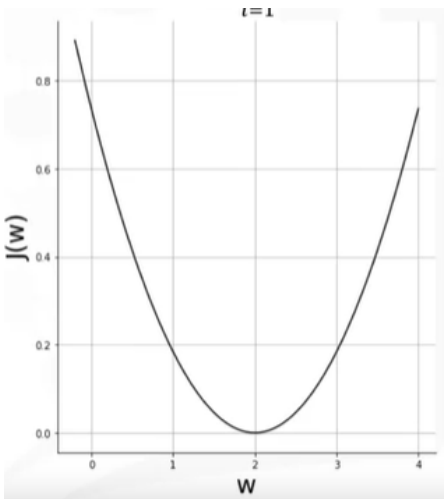
Suppose we want to find the value of **w** to generate a line that best fits the data. To do that, we define **Cost** or **Loss Function**.



$$Z = wX$$

One common **Cost** or **Loss Function** is shown below…

$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(z_i - wx_i)^2$$



The best thing about this function is that it is a parabola, and has one global minimum or one unique solution.

This is a very simplified example as in the real-world datasets, the target variable **z** would depend on more than one variable. We can't simply plot the cost function and visually determine the best value of the weights.

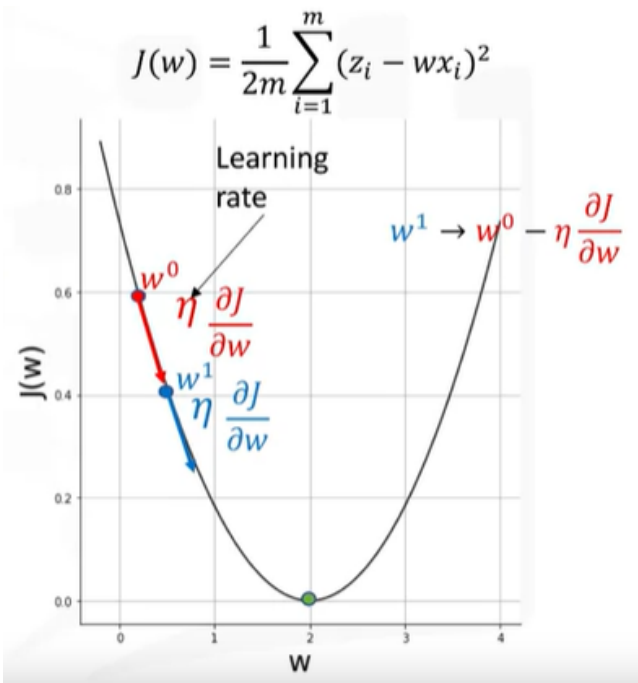
Gradient Descent Algorithm

The best value of **w**, or **w**'s, if we have many weights to optimize, is determined through an algorithm called **Gradient Descent**. It is an iterative optimisation algorithm for finding the minimum of the function.

To do so, we start at a random initial value of **w** calling it $\mathbf{w_0}$. Say it is equal to 0.2, and we start taking steps towards the **green dot** which is equal to **w = 2**.

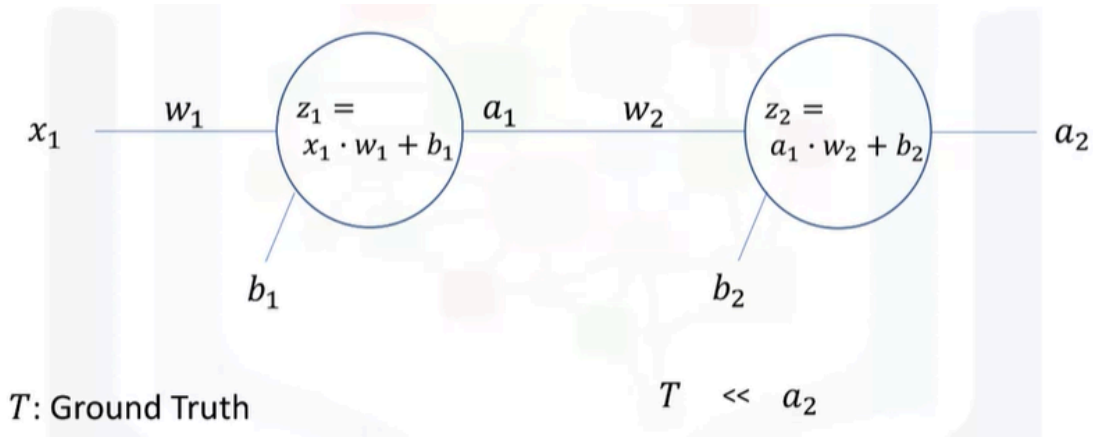
To determine in which direction to move, we compute the gradient of the **Loss function** at the current value of **w**, which is 0.2. The gradient is given by the slope of the tangent at **w = 0.2**, and then the magnitude of the step is controlled by a parameter called the **Learning Rate**. The larger the **Learning Rate**, the bigger the step we take, and the smaller the **Learning Rate**, the smaller the step we take. Then we take and we move to $\mathbf{w_1}$. It is essentially computed as $\mathbf{w_0}$ minus the **Learning Rate** times the gradient at $\mathbf{w_0}$. This represents the first iteration of the algorithm. We repeat this until we get or reach the minimum.

While choosing the **Learning Rate**, we should be careful because a larger rate leads to larger steps and might miss the minimum value. A smaller rate might lead to small steps making the algorithm take more time to reach the minimum point.



Backpropagation

How do Neural Networks train and optimize their weights and biases for a given problem and data set?



Training is done in a supervised learning setting, where each data point has a corresponding label or **Ground Truth**. The training is needed when the predicted value doesn't match the **Ground Truth** for the given input.

Backpropagation Algorithm

1. Calculate the error between the Ground Truth and the estimated output. Let's denote the error by **E**.
2. Propagate the error back into the network and update each weight and bias as per the following equations:

$$w_i \rightarrow w_i - \eta \cdot \frac{\partial E}{\partial w_i}$$

$$b_i \rightarrow b_i - \eta \cdot \frac{\partial E}{\partial b_i}$$

We calculate the squared error between the Ground Truth **T** and the Predicted Value $\mathbf{a_2}$. This represents our **Cost** or **Lost Function**.

$$E = \frac{1}{2}(T - a_2)^2$$

Since there is not only a single point but thousands, we calculate the **Mean Squared Error**.

$$E = \frac{1}{2m}\sum_{i=1}^{m}(T_i - a_{2,i})^2$$

Then we use the error to update $\mathbf{w_2}$, $\mathbf{b_2}$, $\mathbf{w_1}$ and $\mathbf{b_1}$ by propagating the error back into the network.

Updating $\mathbf{w_2}$

By using the equations,

$$w_2 \rightarrow w_2 - \eta \cdot \frac{\partial E}{\partial w_2}$$

$$E = \frac{1}{2}(T - a_2)^2 \longrightarrow \frac{\partial E}{\partial a_2}$$

$$a_2 = f(z_2) = \frac{1}{1 + e^{-z_2}} \longrightarrow \frac{\partial a_2}{\partial z_2}$$

$$z_2 = a_1 \cdot w_2 + b_2 \longrightarrow \frac{\partial z_2}{\partial w_2}$$

$$w_2 \rightarrow w_2 - \eta \cdot \frac{\partial E}{\partial w_2}$$

$$\frac{\partial E}{\partial w_2} = \frac{\partial E}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2}$$

$$= \big(-(T - a_2)\big) \cdot \big(a_2(1 - a_2)\big) \cdot (a_1)$$

$$w_2 \rightarrow w_2 - \eta \cdot \big(-(T - a_2)\big) \cdot \big(a_2(1 - a_2)\big) \cdot (a_1)$$

Updating b_2 (Similar to w_2)

$$b_2 \rightarrow b_2 - \eta \cdot \frac{\partial E}{\partial b_2}$$

$$\frac{\partial E}{\partial b_2} = \frac{\partial E}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2}$$

$$= \left(-(T - a_2)\right) \cdot \left(a_2(1 - a_2)\right) \cdot 1$$

$$b_2 \rightarrow b_2 - \eta \cdot \left(-(T - a_2)\right) \cdot \left(a_2(1 - a_2)\right) \cdot 1$$

Updating w_1 (Similar to w_2)

$$w_1 \rightarrow w_1 - \eta \cdot \frac{\partial E}{\partial w_1}$$

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

$$= \left(-(T - a_2)\right) \cdot \left(a_2(1 - a_2)\right) \cdot (w_2) \cdot a_1(1 - a_1) \cdot x_1$$

$$w_1 \rightarrow w_1 - \eta \cdot \left(-(T - a_2)\right) \cdot \left(a_2(1 - a_2)\right) \cdot (w_2) \cdot a_1(1 - a_1) \cdot x_1$$

Updating b_1 (Similar to w_2)

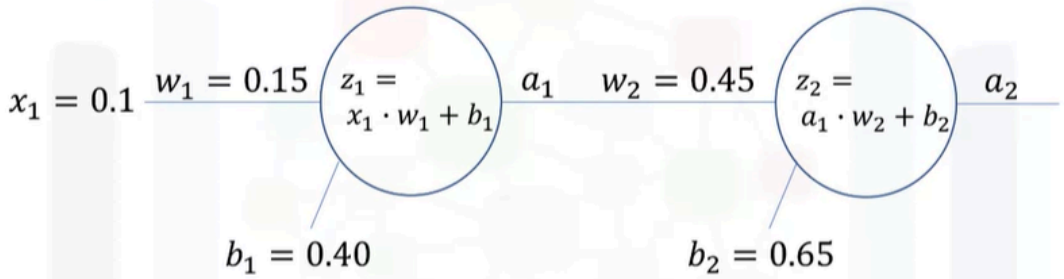
$$b_1 \rightarrow b_1 - \eta \cdot \frac{\partial E}{\partial b_1}$$

$$\frac{\partial E}{\partial b_1} = \frac{\partial E}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1}$$

$$= \left(-(T - a_2)\right) \cdot \left(a_2(1 - a_2)\right) \cdot (w_2) \cdot a_1(1 - a_1) \cdot 1$$

$$b_1 \rightarrow b_1 - \eta \cdot \left(-(T - a_2)\right) \cdot \left(a_2(1 - a_2)\right) \cdot (w_2) \cdot a_1(1 - a_1) \cdot 1$$

Let's Apply the above to an example.



$$z_1 = w_1 \cdot x_1 + b_1$$
$$= 0.15 \cdot 0.1 + 0.4$$
$$= 0.415$$

$$z_2 = w_2 \cdot a_1 + b_2$$
$$= 0.45 \cdot 0.6023 + 0.65$$
$$= 0.9210$$

$$a_1 = f(z_1) = \frac{1}{1+e^{-(0.415)}} = 0.6023$$

$$a_2 = f(z_2) = \frac{1}{1+e^{-(0.9210)}} = 0.7153$$

Let's assume,

$$z_1 = 0.415$$
$$a_1 = 0.6023$$
$$z_2 = 0.9210$$
$$a_2 = 0.7153$$

$$T = 0.25$$

$$E = \frac{1}{2}(T - a_2)^2$$
$$= \frac{1}{2}(T - a_2)^2 = 0.1083$$

$$epochs = 1000$$
$$\epsilon = 0.001$$
$$\eta = 0.4$$

Update $\mathbf{w_2}$,

$$w_2 \rightarrow w_2 - \eta \cdot \left(-(T - a_2)\right) \cdot \left(a_2(1 - a_2)\right) \cdot (a_1)$$

$$w_2 \rightarrow 0.45 - 0.4 \cdot \left(-(0.25 - 0.7153)\right) \cdot \left(0.7153(1 - 0.7153)\right) \cdot (0.6023)$$

$$w_2 \rightarrow 0.45 - 0.4 \cdot 0.05706$$

$$w_2 \rightarrow 0.427$$

Update $\mathbf{b_2}$,

$$b_2 \rightarrow b_2 - \eta \cdot (-(T - a_2)) \cdot (a_2(1 - a_2)) \cdot 1$$

$$b_2 \rightarrow 0.65 - 0.4 \cdot (-(0.25 - 0.7153)) \cdot (0.7153(1 - 0.7153)) \cdot 1$$

$$b_2 \rightarrow 0.65 - 0.4 \cdot 0.0948$$

$$b_2 \rightarrow 0.612$$

Update $\mathbf{w_1}$,

$$w_1 \rightarrow w_1 - \eta \cdot (-(T - a_2)) \cdot (a_2(1 - a_2)) \cdot (w_2) \cdot a_1(1 - a_1) \cdot x_1$$

$$w_1 \rightarrow 0.15 - 0.4 \cdot (-(0.25 - 0.7153)) \cdot (0.7153(1 - 0.7153)) \cdot 0.45 \cdot 0.6023(1 - 0.6023) \cdot 0.1$$

$$w_1 \rightarrow 0.15 - 0.4 \cdot 0.001021$$

$$w_1 \rightarrow 0.1496$$

Update $\mathbf{b_1}$,

$$b_1 \rightarrow b_1 - \eta \cdot (-(T - a_2)) \cdot (a_2(1 - a_2)) \cdot (w_2) \cdot a_1(1 - a_1) \cdot 1$$

$$b_1 \rightarrow 0.40 - 0.4 \cdot (-(0.25 - 0.7153)) \cdot (0.7153(1 - 0.7153)) \cdot 0.45 \cdot 0.6023(1 - 0.6023) \cdot 1$$

$$b_1 \rightarrow 0.40 - 0.4 \cdot 0.01021$$

$$b_1 \rightarrow 0.3959$$

This was our 1st iteration. With the updated weights and biases, we will perform **Forward Propagation** calculate the **Predicted Values,**

and compare them with the **Ground Truth**. Calculate the error and do another round of **Backpropagation**.

Complete Training Algorithm

1. Initialize the weights and the biases.
2. Iteratively repeat the following steps:
    - Calculate network output using forward propagation.
    - Calculate the error between ground truth and estimated or predicted output
    - Update weights and biases through backpropagation
    - Repeat the above 3 steps until the number of iterations/epochs is reached or the error between ground truth and predicted output is below a predefined threshold.

Vanishing Gradient Problem

Since we are using the **Sigmoid Function** as the **Activation Function**, all the intermediate values in the network are between 0 and 1. During the **Backpropagation**, we keep multiplying factors that are less than 1 by each other, so their gradients get smaller and smaller as we keep moving backwards in the network.

It means that the neurons in the earlier layers learn very slowly as compared to the neurons in the later layers of the network. The result is a training process that takes too long and a prediction accuracy that is compromised.

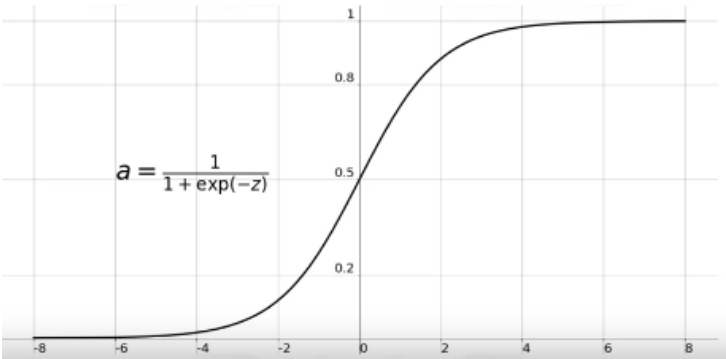
This is the reason we do not use the **Sigmoid Function** or a similar function as the **Activation Function** since they are prone to this problem.

Activation Functions

There are 7 types of Activation Functions:

1. Binary Step Functions
2. Linear/Identity Function
3. Sigmoid/Logistic Function
4. Hyperbolic Tangent Function
5. ReLU (Rectified Linear Unit) Function
6. Leaky ReLU Function
7. Softmax Function

Sigmoid Function

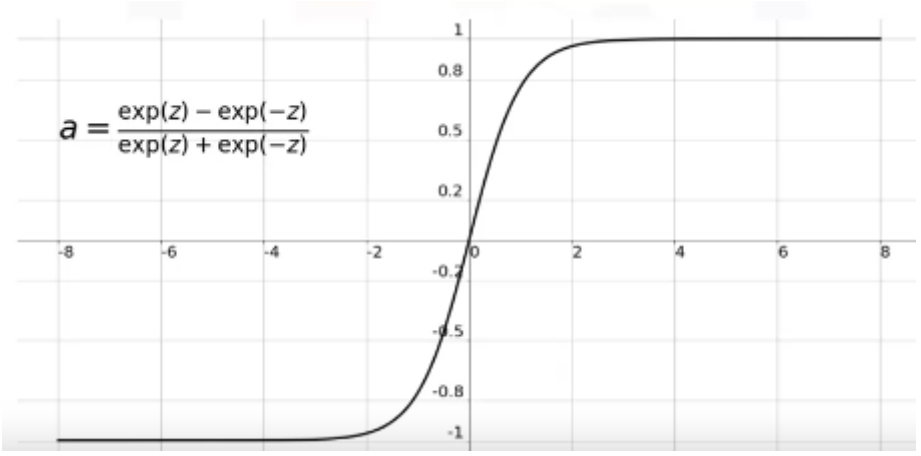


When **z** is a large positive number, the result is close to 1. When **z** is a large negative number, the result is close to 0.

As we can see the graph becomes **flat** at the larger values, therefore we say that the neural network does not learn at this point.

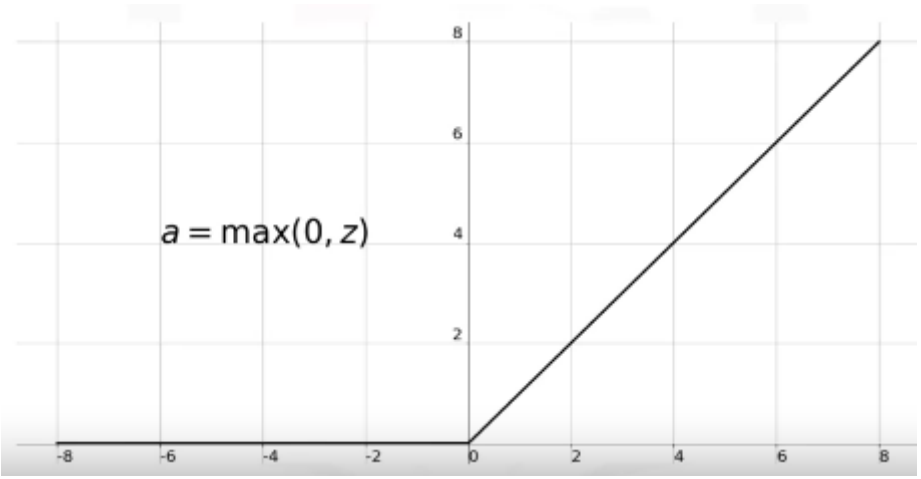
Another problem with this function is that the output always ranges from 0 to 1 meaning that it is always positive. It's not always necessary that the input to the next neuron always be of the same sign.

Hyperbolic Tangent Function



It is very similar to the Sigmoid Function (scaled version of the Sigmoid Function). Unlike the Sigmoid Function, it is symmetric over the origin. It ranges from **-1** to **+1**. Although it is better than the Sigmoid Function, it also leads to the **Vanishing Gradient** problem in very deep neural networks.

ReLU Function



It is the most widely used **Activation Function** when designing networks today. In addition to being non-linear, the main advantage of this function is that it does not activate all the neurons at the same time.

According to the plot, if the input is negative, it is converted to **0** and the neuron does not get activated. This means that only a few neurons are activated making the network more sparse and efficient. It was one of the main advancements in the field of **Deep Learning** that led to overcoming the **Vanishing Gradient** problem.

Softmax Function

$$a_i = \frac{e^{z_i}}{\sum_{k=1}^{m} e^{z_k}}$$

It is also a type of Sigmoid Function but, it is much more handy when we are trying to handle classification problems. It is ideally used in the output layer of the classifier where we are trying to get the probabilities to define the class of each input.

$$z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} 1.6 \\ 0.55 \\ 0.98 \end{bmatrix}$$

$$\Big\downarrow \text{Softmax}$$

$$a = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0.51 \\ 0.18 \\ 0.31 \end{bmatrix}$$

It is easier for us to classify a given data point and determine to which category it belongs.

Generally, we can begin using the ReLU Activation Function and then switch to other Activation Functions if ReLU doesn’t yield optimum results.

Deep Learning Libraries

Popular libraries that are used in Deep Learning are:

1. **TensorFlow**
2. **Keras**
3. **PyTorch**
4. **The** Montreal Institute developed Theano and is even older than others. However, it is no longer supported.

Keras vs **PyTorch** vs **TensorFlow**

- **TensorFlow** is the most popular deep-learning library, developed by **Google**.
- **PyTorch** is a cousin of the Lua-based Torch framework and is a strong competitor of **TensorFlow**.
- **Keras** is the easiest API to use and the go-to library for quick prototyping and fast development.

Regression Model with Keras

	Cement	Blast Furnace Slag	Fly Ash	Water	Superplasticizer	Coarse Aggregate	Fine Aggregate	Age	Strength
0	540.0	0.0	0.0	162.0	2.5	1040.0	676.0	28	79.99
1	540.0	0.0	0.0	162.0	2.5	1055.0	676.0	28	61.89
2	332.5	142.5	0.0	228.0	0.0	932.0	594.0	270	40.27
3	332.5	142.5	0.0	228.0	0.0	932.0	594.0	365	41.05
4	198.6	132.4	0.0	192.0	0.0	978.4	825.5	360	44.30

The above dataset contains the compressive strengths of the concrete samples based on the volumes of materials that were used to make them.

The **Deep Neural Network** that we would like to connect takes all the features as input and feeds them into a **hidden layer** of 5 nodes, which is connected to another **hidden layer** of 5 nodes, which is then connected to the **Output Layer** with 1 node, which is supposed to give a single output, i.e., the Compressive Strength.

Usually, we use a Network with a large number of nodes in a layer of about 100 or more; such a network is called a **dense network**.

Before using the **Keras** library, we need to format our dataset correctly. We will need to split our dataframe into 2 dataframes, one has the **predictors** column and another one has the **target** column. We will name them **Predictors** and **Target**.
Using the **Keras** library we can achieve this by just a few lines of code.

```
import keras
from keras.models import Sequential
from keras.layers import Dense

model = Sequential()

n_cols = concrete_data.shape[1]

model.add(Dense(5, activation='relu', input_shape=(n_cols,)))
model.add(Dense(5, activation='relu'))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(predictors, target)

predictions = model.predict(test_data)
```

Some of the topics in detail are:-

- Keras Activation Function: https://keras.io/activations/
- Keras Models: https://keras.io/models/about-keras-models/#about-keras-models
- Keras Optimizers: https://keras.io/optimizers/
- Keras Metrics: https://keras.io/metrics/

Classification Models with Keras

	price_high	price_low	price_med	maintenance_high	maintenance_low	maintenance_med	persons_2	persons_more	decision
0	1	0	0	1	0	0	1	0	0
1	1	0	0	1	0	0	1	0	0
2	1	0	0	1	0	0	1	0	0
3	1	0	0	1	0	0	1	0	0
4	1	0	0	1	0	0	1	0	0

The above dataset describes purchasing a certain car based on the purchase price, maintenance cost, personal accommodation etc. Look at the dataset properly.

$$\text{decision} = \begin{cases} 0: & \text{Not acceptable} \\ 1: & \text{Acceptable} \\ 2: & \text{Good} \\ 3: & \text{Very Good} \end{cases}$$

Here, we will use the same Neural Network that we have used in the Regression Model previously. The network takes 8 inputs or predictors and consists of 2 hidden layers each of 5 neurons, and an **Output Layer**. We will divide our dataset into **Predictors** and **Targets**.

In **Classification Models**, we can't use the target column as is, we need to transform the column into an array with binary values similar to **One-Hot Encoding**. We can achieve that by using the "to_categorical" function from the **Keras**.

decision
0
0
0
0
0

target

```
target = to_categorical(car_data['decision'])
target
```

```
array([[1., 0., 0., 0.],
       [1., 0., 0., 0.],
       [1., 0., 0., 0.],
       ...,
       [1., 0., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.]], dtype=float32)
```

Instead of having a single **Output Neuron**, our model will have 4 **Output Neurons** since we have 4 target variables. Our code will be almost the same as that of the **Regression Model**.

```python
import keras
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import to_categorical

model = Sequential()

n_cols = car_data.shape[1]

target = to_categorical(target)

model.add(Dense(5, activation='relu', input_shape=(n_cols,)))
model.add(Dense(5, activation='relu'))
model.add(Dense(4, activation='softmax'))

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(predictors, target, epochs=10)
```

```python
model.predict(test_data)
```

Keras Predict Method

The output of the prediction method of Keras will be like this…

```python
model.predict(test_data)
```

```
array([[9.9978679e-01, 2.1028408e-04, 1.0243932e-06, 1.8633460e-06],
       [9.9978679e-01, 2.1028408e-04, 1.0243932e-06, 1.8633460e-06],
       [9.9978679e-01, 2.1028408e-04, 1.0243932e-06, 1.8633460e-06],
       ...,
       [4.9434581e-01, 4.9560347e-01, 4.4486104e-03, 5.6021004e-03],
       [4.9434581e-01, 4.9560347e-01, 4.4486104e-03, 5.6021004e-03],
       [4.9434581e-01, 4.9560347e-01, 4.4486104e-03, 5.6021004e-03]],
      dtype=float32)
```

Each data point is the probability of “Purchasing a Car” belonging to one of the four classes. For each data point, the sum of probability should be 1. The higher the probability, the more confidence that the data point belongs to a particular class.

Shallow vs Deep Neural Networks

Shallow Deep Neural Network

- A shallow neural network consists of one hidden layer.
- A deep neural network consists of more hidden layers and has a large number of neurons in each layer.

Unlike **Shallow Neural Networks** which take only input as vectors, **Deep Neural Networks** can take raw data such as images and text and automatically extract the features to learn the data better.

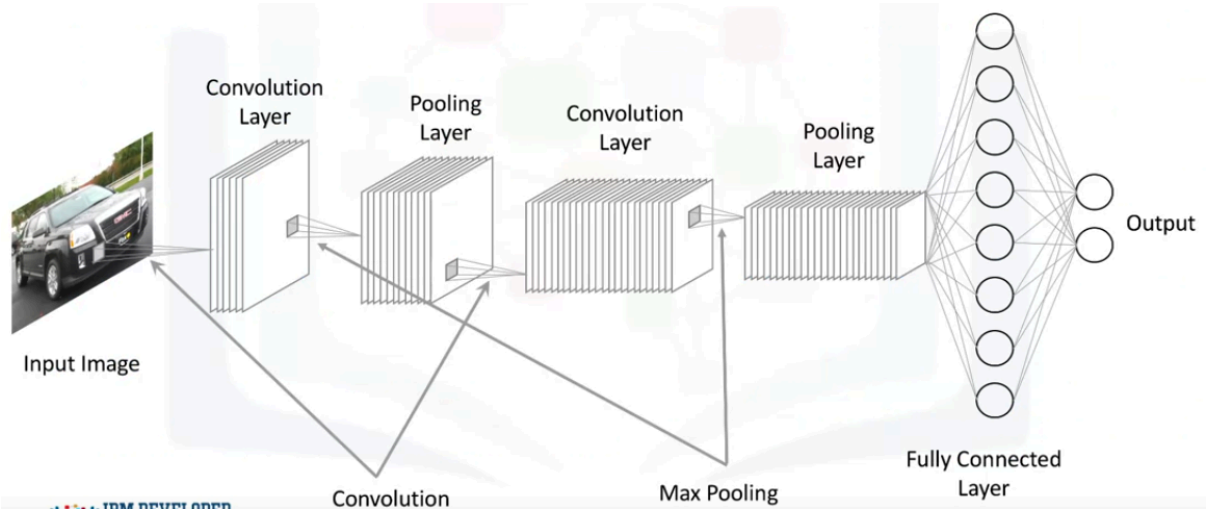
Why Deep Learning Took off

1. Advancement in the field itself.
2. Data
3. Computational Power - GPU Power!

Convolutional Neural Networks

CNNs are similar to the typical neural networks. **CNN**s take inputs as images. This allows us to incorporate properties that make the training process much more efficient. Solve problems involving image recognition, object detection, and other **Computer Vision Applications**.

CNN Architecture



The network consists of a series of **Convolutional**, **ReLU**, and **Pooling Layers** as well as several fully connected layers which are necessary before the output is generated. Let's see what happens in each layer…

Input Layer

We have dealt with **Conventional Neural Networks** that take an **(n x 1)** vector as their input.

The input to the **Convolutional Neural Network**, on the other hand, is mostly an **(n x m x 1)** for **Grayscale Images** or **(n x m x 3)** for **Colored Images**, where the number 3 represents the **RGB Components** of each pixel in the image.

Convolutional Layer

Here, we define filters and we compute the **Convolution** between the defined filter and each of the three images. We apply the filter by sliding the **Filter** over the image computing the **Dot Product**

between the filter and the **Overlapping Pixel** value and storing the result in the empty matrix. We repeat this process, moving **One Stride** at a time until we cover the entire image.

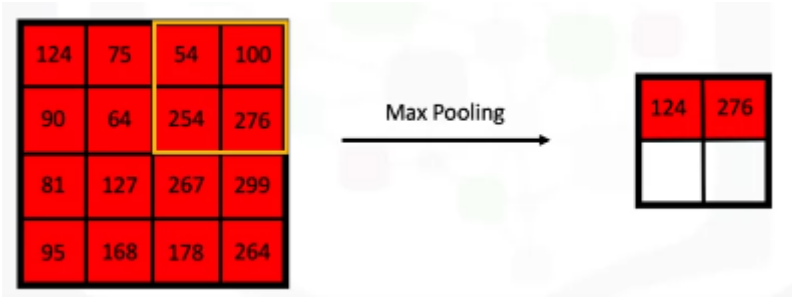
The same will be applied to the **Green** and **Blue** images and we can apply more than one filter. The more filters we use, the more we can preserve the **Spatial Dimensions**.

Remember: We don't flatten the dimensions because we will end with a massive number of parameters that will need to be optimized, and will be super computationally expensive. Also decreasing the number of parameters will help prevent the model from overfitting. This layer uses the **ReLU Layer** passing only the positive inputs.

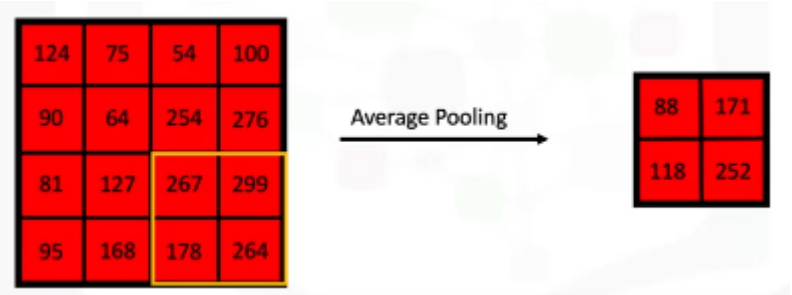
Pooling Layer

Its main objective is to reduce the Spatial Dimensions of the data propagating through the network. There are 2 types of Pooling i.e. Max pooling and Average Pooling.

In **Max Pooling**, for each section, we scan we keep the highest value.



In **Average Pooling**, we compute the average of each area we scan.



This method provides **Spatial Variance** which enables the neural network to recognize the image even if the object does not exactly resemble the original object.

Fully-Connected Layer

Here, we flatten the output of the last **Convolutional Layer** and connect every node of the current layer with every other node of the next layer. This layer takes the output from the preceding layer as the input and outputs and an **N-dimensional vector** where **N** is the number of classes.

If we are building a network to classify the images of digits, then the dimension **n** would be 10. Since there are 10 digits.

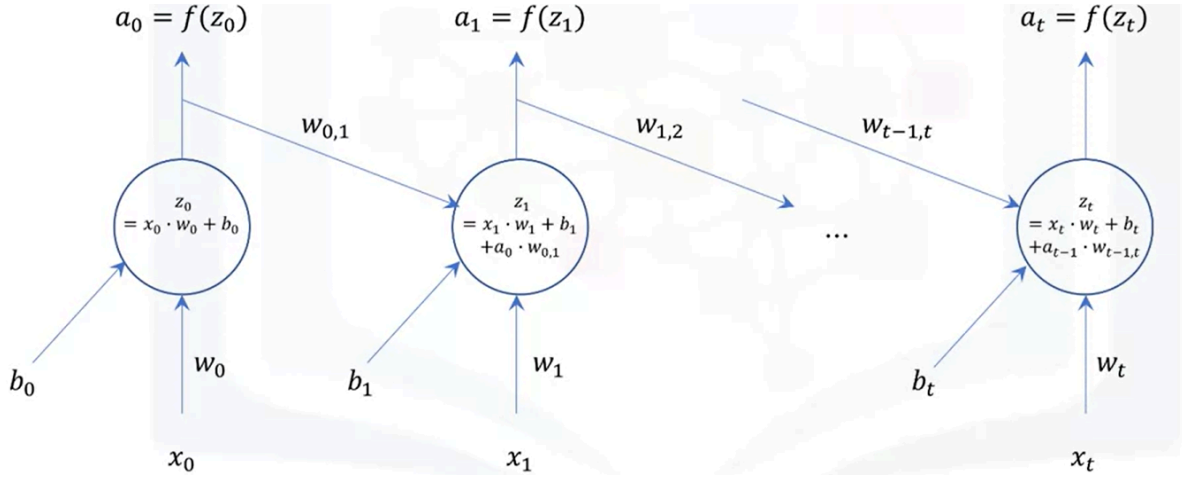
Keras Code

```
model = Sequential()

input_shape = (128, 128, 3)

model.add(Conv2D(16, kernel_size=(2, 2), strides=(1, 1),
                 activation='relu',
                 input_shape=input_shape))

model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))
model.add(Conv2D(32, kernel_size=(2, 2), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(100, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))
```

Recurrent Neural Networks

Let's say we want to build a model that can analyse scenes in a movie. We cannot assume that scenes in a film are independent; therefore, traditional **deep-learning** models are unsuitable for this application.

Recurrent Neural Networks (RNNs) overcome this issue. They are networks with loops that don't just take a new input at a time but also take in input the output from the previous data point that was fed into the network.



We can start with a **Normal Neural Network**. At time **t = 0**, the network takes in input $\mathbf{x_0}$ and outputs $\mathbf{a_0}$. Then, at time **t = 1**, in addition to the input $\mathbf{x_1}$, the network also takes $\mathbf{a_0}$ as input, weighted with weight $\mathbf{w_0, 1}$, and so on and so forth.

As a result, **RNNs** are very good at modelling patterns and sequences of data such as texts, genomes, handwriting, and stock markets. These algorithms take time and sequence into account, which means that they have a temporal dimension.

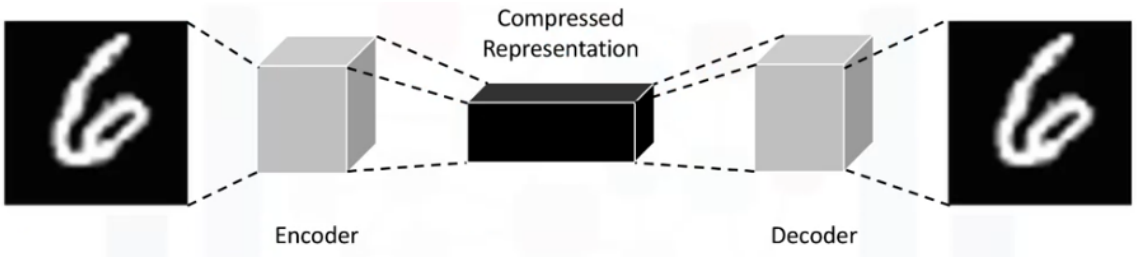
Long Short-Term Memory Model (LSTM)

It is a popular example of an RNN. It has many successful applications like:

- Image Generation
- Handwriting Generation
- Automatically Captioning of Images
- Automatic Description of Videos

Autoencoders

It is a data compression algorithm where the compression and decompression functions are learned automatically from data. They are data-specific. Some interesting applications are data denoising and dimensionality reduction from data visualisation.



It takes an image, for example, as an input and uses an **Encoder** to find the optimal compressed representation of the input image. Then, using a **Decoder** the original image is restored.

So an **Autoencoder** is an **Unsupervised Neural Network** model. It tries to predict **x** from **x** without the need for any labels. They can learn data projections that are more interesting than **Principal Component Analysis (PCA)** or other basic techniques which can handle only **Linear Transformations**.

Restricted Boltzmann Machines (RBMs)

It is a trendy type of Autoencoder. It has various successful applications:

- Fixing Imbalanced Datasets
- Estimating Missing Values
- Automatic Feature Extraction